

UNIVERSITY OF SOUTHERN DENMARK

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

SPDM801: MASTER THESIS IN COMPUTER SCIENCE

Formalising π LL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

1st June 2026



Abstract

This thesis presents a mechanised formalisation of π LL, a process calculus relating linear logic and session-typed communication, in Lean 4. The formalisation covers the static structure of the full calculus, including session types, processes, environments, hyperenvironments, typing judgements, and substitution principles. To represent binding, the development retains explicit free identifiers while encoding bound occurrences by indices: channel binders are handled through a locally nameless representation with co-finite premises, while polymorphic type binders are treated through type-depth indexing, shifting, and substitution. The operational semantics and session-fidelity development are mechanised for the multiplicative fragment, π MLL, using a derivation-level transition system together with projected process-level and environment-level relations. The completed metatheory establishes the projection of typed transitions to process and resource transitions and develops the reverse lifting argument for all multiplicative cases except the restriction case. The remaining *res* case is isolated to the reconstruction of endpoint-bearing hyperenvironment components after a transition under restriction. The development demonstrates that mechanising the paper theory in this way can relocate implicit reasoning about bound names, freshness, and unordered linear resources into explicit structural invariants. It thereby contributes a mechanised account of π LL and identifies the remaining infrastructure required to complete session fidelity for π MLL.

Resumé

Denne afhandling præsenterer en mekaniseret formalisering af π LL, en proceskalkule, der forbinder lineær logik med sessionstypet kommunikation, i Lean 4. Formaliseringen dækker den statiske struktur af den fulde kalkule, herunder sessionstyper, processer, miljøer, hypermiljøer, *typing judgements* og substitutionsprincipper. Til repræsentation af binding fastholder udviklingen eksplicitte frie navne, mens bundne forekomster repræsenteres ved indeks: kanalbindere håndteres gennem en *locally nameless* repræsentation med co-finite-præmisser, mens polymorfe typebindere behandles gennem typedybdeindeksering, forskydning og substitution. Den operationelle semantik og udviklingen af *session fidelity* er mekaniseret for det multiplikative fragment, π MLL, ved hjælp af et transitionssystem over *typing derivations* samt projicerede transitionsrelationer for processer og hypermiljøer. Den gennemførte metateori etablerer projektionen af typede transitioner inkluderende til proces- og ressourcetransitioner og udvikler det omvendte løfteargument for alle multiplikative tilfælde undtagen restriktionstilfældet. Det resterende *res*-tilfælde er isoleret til rekonstruktionen af de hypermiljøkomponenter, der bærer de restrikerede endepunkter, efter en transition under restriktion. Udviklingen viser, at en mekanisering af papirteorien på denne måde kan flytte implicit ræsonnement om bundne navne, friskhed og uordnede lineære ressourcer til eksplicitte strukturelle invarianter. Afhandlingen bidrager dermed med en mekaniseret redegørelse for π LL og identificerer den resterende infrastruktur, der kræves for at fuldføre sessionsfidelitet for π MLL.

CONTENTS

1	Future Work	3
1.1	Extending to full π LL	3
1.2	Structural congruence	4
1.3	Readiness, productivity and behavioural metatheory	5
	References	5

1 Future Work

The most immediate directions for future work concern completing the remaining session-fidelity proof obligation and extending the dynamic metatheory to the full π LL calculus.

subsectionCompleting Session Fidelity for π MML Completing session fidelity for the multiplicative fragment requires both closing the remaining proof gap and reducing the proof-engineering overhead that made this case difficult. The following directions concern the missing restriction case, a possible reformulation of process transitions, and automation of the recurring structural obligations.

Completing the remaining proof case. The immediate remaining proof obligation is the `res` case of `typability_subject_reductionm`. As established in ??, completing this case requires recovering the two endpoint-bearing hyperenvironment components after applying the induction hypothesis beneath a restriction. The current proof attempt isolates this requirement into two auxiliary results: a single-block preservation lemma, needed when only one distinguished component lies beneath a stepping parallel operand, and a paired-endpoint reconstruction lemma, needed to recover the two dual components consumed by the enclosing cut. The remaining work lies in completing their permutation-sensitive cases for parallel propagation, synchronisation, internal communication, and nested restriction transitions.

Co-finite quantification in ProcStep. The current `ProcStepm` relation records concrete fresh names in rules that open bound continuations, while the corresponding typing rules use co-finite quantification. This asymmetry introduces proof overhead in the typability argument, where the concrete names chosen by a process transition must be reconciled with the co-finite premises of the typing derivation. Refactoring the relevant process-transition rules, including `tensor`, `parr`, `one_bot`, `tensor_parr`, and `res`, to use a co-finite formulation could align the two relations more directly and reduce the need for auxiliary *all-fresh* lemmas. Such a reformulation would, however, make the erased process semantics less directly concrete, and its effect on the erasure simulation proofs remains to be investigated.

Proof automation. A recurring cost throughout the formalisation is the manual rearrangement of hyperenvironments using permutation, disjointness, name-set, and freshness lemmas. Many of these obligations amount to routine framing arguments: showing that components remain disjoint, that two hyperenvironments agree up to permutation, or that a chosen name is fresh for the relevant context. A dedicated `aesop` rule set, together with a curated collection of simplification lemmas for normalising name-set and disjointness goals, could automate much of this bookkeeping and significantly reduce proof size. This would not replace the main inversion and reconstruction arguments, such as the remaining `res` case, but could make those arguments substantially more manageable.

1.1 Extending to full π LL

Before extending to the full calculus, it may be worth first aligning the freshness disciplines used across the transition systems, specifically transitioning `ProcStepm` to use co-finite quantification like `TypingStepm` and the static `Typing` relation and ensuring an even formulation across the three relations. Once the multiplicative fragment has been consolidated in this way, extending to full π LL should follow as a natural next step.

Once the multiplicative fragment is complete, extending the formalisation to the full π LL calculus appears to require additional proof engineering rather than a new formal architecture. The static model is already defined for the full calculus, and many typing inversion lemmas for the additive, polymorphic, axiom, and exponential constructs are already available. The main work

is therefore expected to lie in extending $\text{typability_subject_reduction}_m$ and in proving the hyperenvironment extraction lemmas needed for the additional synchronisation cases, rather than in the erasure projections or the unit-like transition axioms. Some additional work would also be needed to extend the operational semantics with the transition constructors corresponding to the full set of rules from Appendix B.2 of Montesi and Peressotti [2021].

The remaining prefix and unit-like transition axioms from Appendix B.2 of Montesi and Peressotti [2021] are expected to follow the same proof pattern as the already mechanised one and bot cases. This includes the additive selections, server-use actions, polymorphic send and receive actions, disposal and duplication send actions, and the link actions. In each case, the proof should proceed by inverting the source typing derivation, constructing the corresponding typed transition, and reusing the existing local-closure and freshness infrastructure. The polymorphic cases additionally rely on the type-substitution and shifting lemmas developed for the locally nameless representation, while the link and axiom-cut cases require the name-substitution lemmas. The axiom-cut interaction may require special treatment, since it combines restriction with name substitution; one possible approach is to add an explicit axiom-under-restriction transition constructor, analogous to the existing one_bot and tensor_parr synchronisation constructors.

The remaining synchronisation cases, such as $!w$, $!c$, $!?$, $\exists V$, $\oplus_1 \&$, and $\oplus_2 \&$, follow the same general restriction-synchronisation pattern as the multiplicative communication cases. They require isolating the interacting components, stepping the opened body, and reconstructing the resulting hyperenvironment up to permutation. Once the extraction lemmas for the current res case are completed, these cases are expected to follow by analogous reasoning, with the relevant connectives and environments substituted for their multiplicative counterparts.

The duplication and disposal receive cases are more involved than the other server actions, since they generate a larger continuation process and manipulate dependencies and the typing environment. A prior revision of the formalisation, where server dependencies were stated implicitly, included the auxiliary constructions for building these processes and manipulating the corresponding dependencies and typing environments. Since the well-typedness of those constructions had already been established, parts of this earlier infrastructure may be reusable when extending the current dynamic metatheory.

1.2 Structural congruence

A further direction is to mechanise process-level structural congruence and prove that it preserves typing and transitions. This would address the artificial stuck states that arise from the strict binary-tree structure of the process syntax. In particular, commutativity and associativity of parallel composition are needed for the syn rule to apply to processes that are not immediate siblings in the syntax tree, as discussed in ?? . A structural congruence relation would internalise the commutative, associative, and unit laws of parallel composition, as well as scope manipulation for restriction, rather than requiring these rearrangements to be handled manually through exchange rules and hyperenvironment permutation lemmas.

Scope manipulation is especially delicate in the locally nameless setting, since moving a process under a restriction binder requires explicit de Bruijn index management. For example, a scope-extrusion principle of the form $(\nu(\bullet, \bullet) P) \mid Q \equiv \nu(\bullet, \bullet) (P \mid Q')$ would require Q' to be obtained by shifting the bound channel indices of Q by 2, while leaving its free names unchanged, to account for the two additional channel binders introduced by the restriction. This would require both a channel-level shift operation on bound indices and a process-level traversal lifting that operation through the syntax.

The tradeoff is between modularity and local proof convenience. A full structural congruence relation would require preservation theorems showing that typing and transitions are stable under

each congruence principle, but these results would then be reusable throughout the metatheory. A rule-oriented approach instead adds symmetric variants of selected rules, or selected symmetry and reassociation principles, directly to the transition relation. This could account for the congruences most relevant to stepping while preserving the syntax-directed structure of the system, which is useful for proof search in Lean. However, these additional constructors would still introduce extra cases in later proofs. Although unordered collections of processes could avoid some commutativity issues, representing process syntax quotient-wise would make syntax-directed inversion and induction substantially harder in Lean, mirroring the difficulties that led environments to be represented as lists rather than finite sets.

1.3 Readiness, productivity and behavioural metatheory

Beyond session fidelity, the operational metatheory of π LL includes results concerning the availability of interactions, the behaviour of infinite executions, and relations between concurrent process behaviours.

Readiness. One of the unmechanised metatheoretic objectives identified in the project description is progress. In the theory of Montesi and Peressotti [2021], this objective corresponds to the readiness theorem. Theorem 5.20 states that, whenever $\vdash P :: \Gamma_1 \mid \cdots \mid \Gamma_n$, every component Γ_i contains some name x such that P can perform an action whose free names include x . Thus, readiness states that each component of the typed interface of a process offers at least one available interaction. Formalising this result would require proving that, for every component of a well-typed process's hyperenvironment, the process can currently perform a labelled transition involving a channel described by that component. The appropriate Lean proof architecture for establishing this correspondence remains to be developed.

Productivity and potential. The paper additionally proves productivity, which concerns the behaviour of infinite executions. Theorem 5.19 states that every infinite execution of a well-typed process contains infinitely many non-silent transitions. Equivalently, a well-typed process cannot execute internally forever while performing only finitely many observable actions. The proof uses a potential-annotated typing system: Lemma 5.18 bounds the length of any finite sequence of τ -transitions by the potential annotation, while Lemma 5.17 connects annotated typing derivations with ordinary typing derivations. Mechanising productivity would therefore require formalising the annotated typing system, its refinement result, and the potential argument used to bound internal computation. The design of this annotated layer and its integration with the present formalisation remain future work.

Further behavioural results. The paper also establishes behavioural and concurrency results that are not mechanised in the present development. These include congruence results for similarity and bisimilarity, the characterisation connecting similarity with typability, and the parallelism properties of the diamond property, serialisation, and non-interference. Similarity and bisimilarity provide relations for comparing process behaviour, while the parallelism properties describe how independent interactions may be reordered or shown not to interfere with one another. Mechanising these results would require introducing the corresponding behavioural relations and extending the transition-system development with the additional metatheoretic arguments from the paper. The organisation and proof-engineering requirements of such an extension remain to be investigated.

References

Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the π -calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818